Intelligent Knowledge Retrieval System

Business Requirements Document

Version 1.0 | April 2026 | **Portfolio Build** | **Upwork Primary**

1. What We Are Building

A production-grade RAG application that lets users query a document knowledge base through a natural language interface. The system combines unstructured document retrieval with structured metadata querying, demonstrating real architectural decision-making rather than a tutorial-style setup.

> The primary goal is a portfolio piece that signals senior-level RAG engineering to Upwork clients and German tech companies. Every architectural decision should be documented and justified — that documentation is the actual portfolio asset.

2. Domain Choice: HR Policy & Compliance Documents

HR policy documents are the right domain for three specific reasons:

Relatable	Every mid-size company has internal policy documents. Instantly understood by Upwork buyers without needing a domain explainer.
Hybrid data	Unstructured PDFs plus structured employee/policy metadata. Forces a real query routing decision: when to hit the vector store vs. the SQL layer.
Germany-relevant	Works-council regulations, Datenschutz, Kurzarbeit, parental leave — signals awareness of the German enterprise market for LinkedIn/full-time attraction.

3. Tech Stack

Ingestion	Python, PyPDF2 / pdfplumber, LangChain text splitters
Embeddings	OpenAI text-embedding-3-small (or sentence-transformers for local demo)
Vector store	Chroma for local dev, Pinecone for hosted demo

Structured data	PostgreSQL with SQLAlchemy ORM
Retrieval logic	LangChain, hybrid BM25 + dense retrieval, cross-encoder re-ranking
LLM	OpenAI GPT-4o
Backend	FastAPI, Dockerised
Frontend	Custom HTML, CSS, JavaScript — no framework. CSS libraries (e.g. Tailwind via CDN) for utility styling. Keeps the stack lightweight while enabling a fully custom, production-grade UI.
Auth	Username + password, JWT tokens, bcrypt hashing
Observability	LangSmith tracing, retrieval score logging per query

Why not n8n or a no-code orchestrator: n8n targets a non-technical buyer segment that clusters around $25-40/hr on Upwork. A Python/LangChain stack signals the engineering depth that justifies $50-65/hr.

Why custom HTML/CSS/JS over a framework: A React or Next.js setup adds build tooling, bundling, and framework overhead that slows iteration on a portfolio demo. Vanilla JS with a CSS utility library (Tailwind via CDN) delivers a fully custom, sleek UI without the setup cost — and keeps the codebase readable to any evaluator regardless of framework familiarity.

4. Auth Decision

Username + password with JWT tokens. Google OAuth adds client secrets setup, domain verification, and consent screen configuration that slows demo setup and adds zero signal for technical buyers. Two seed accounts (admin and standard user) in a setup script is sufficient for portfolio purposes.

5. Stakeholders

Knowledge Manager
Admin

Uploads documents, manages the knowledge base, monitors query logs and system health.

Employee / End User
Standard User

Queries the knowledge base in natural language, views cited source chunks, exports answers.

Portfolio Owner
Builder (Umer)

Architecture decision-maker. The README is written for this stakeholder's narrative.

Upwork Client / Recruiter
Evaluator

Reads the README, browses code, may run the demo. Primary audience for all design decisions.

6. Use Cases and Features

Role	Feature
Admin	Upload PDF documents to the knowledge base — single file or batch
Admin	Delete or update indexed documents
Admin	View query logs with retrieved chunks and relevance scores
Admin	Trigger re-indexing after configuration changes
User	Ask questions in natural language and receive answers with cited source chunks
User	Filter queries by document category or effective date range
User	View conversation history within session
User	Export an answer with its source references as a plain text (.txt) file download
Both	Login and logout with JWT session
Both	Thumbs up / thumbs down feedback on each answer (logged for retrieval evaluation)

7. Data Model (PostgreSQL)

Two tables are stored in PostgreSQL. The full AI response text is not persisted — only the metadata needed for retrieval evaluation and admin auditing.

Column	Type	Notes
— users —		
id	UUID PK	Auto-generated
name	TEXT	Display name shown in admin user list
email	TEXT	Unique — used for login
password_hash	TEXT	bcrypt hashed, never stored plain
role	TEXT	admin or user
status	TEXT	active or suspended
last_active_at	TIMESTAMP	Updated on each successful login
created_at	TIMESTAMP	Auto
— policy_documents —		
id	UUID PK	Auto-generated
file_name	TEXT	Original filename
category	TEXT	e.g. Engineering, Finance, All

Column	Type	Notes
owner_department	TEXT	Department that owns the policy
effective_date	DATE	Used for date-range filtering in queries
chunk_count	INTEGER	Number of chunks indexed into vector store
status	TEXT	Queued / Processing / Indexed / Failed
created_at	TIMESTAMP	Auto
— query_logs —		
id	UUID PK	Auto-generated
user_id	UUID FK	References users table
query_text	TEXT	The user's question as submitted
retrieved_chunk_ids	UUID[]	Array of chunk IDs returned by retrieval
relevance_scores	FLOAT[]	Matching score per chunk, same order as chunk IDs
date_filter_from	DATE	Nullable — set if user applied a date filter
date_filter_to	DATE	Nullable — set if user applied a date filter
feedback	SMALLINT	1 = thumbs up, -1 = thumbs down, 0 = no feedback
created_at	TIMESTAMP	Auto

Full AI response text is not stored. The query log captures what was retrieved and how well — sufficient for evaluation purposes. Storing response text would bloat the database and add no retrieval signal.

8. Build Phases

#	Phase	Deliverable and Notes
1	**Data and ingestion pipeline**	Gather or generate dummy HR PDFs. Build chunking pipeline with configurable strategy: fixed size, sentence-based, recursive. Log chunk counts and sizes for comparison.
2	**Embedding and vector store**	Embed chunks, store in Chroma locally. Set up metadata filtering by document type and date. Document chunking strategy tradeoffs explicitly.
3	**Hybrid retrieval layer**	Combine BM25 sparse retrieval with dense vector retrieval. Add a cross-encoder re-ranker. This is the differentiating architectural decision and must be documented in the README.
4	**Structured data integration**	Two PostgreSQL tables. (1) Policy metadata: document name, owner, effective date, category — used for hybrid SQL + vector query routing. (2) Query log: query text, retrieved chunk IDs, relevance scores, date

#	Phase	Deliverable and Notes
		filter applied, feedback, user ID, timestamp — stored for retrieval evaluation and admin log view. Full AI response text is not stored.
5	**FastAPI backend and auth**	REST endpoints for query, document upload, and admin actions. JWT middleware. Fully Dockerised with a docker-compose setup.
6	**Custom frontend**	HTML/CSS/JS chat interface with source citation display and date-range query filtering. Admin panel pages for document management, upload, user management, query logs, and system health.
7	**Evaluation and README**	Run retrieval precision and recall on a test query set. Document architecture decisions, what was tried, what was dropped and why. This section is the portfolio centrepiece.

9. The One Thing That Must Be Obvious from the Repo

> The retrieval logic is a deliberate architectural choice, not the default path of least resistance. The README must include a section called "Retrieval Design Decisions" that explains why hybrid over pure vector search, why this chunking strategy over the alternatives, and what the measured tradeoff is. That section is the portfolio piece. The code is the evidence.

This is what separates the project from a tutorial follow-along and from the hundreds of generic RAG demos on GitHub. A Upwork client evaluating candidates will spend 90 seconds on the repo. The retrieval design section needs to be the third thing they read.

10. Deployment

All components deploy on free tiers. The full stack is:

Component	Platform	Notes
FastAPI backend	Render	Free web service. Spins down after inactivity — first request after idle takes ~30 seconds. Document this in the README so demo viewers are not surprised.
PostgreSQL	Render	Free managed instance. Expires after 90 days. Keep a seed script in the repo so the database can be recreated with one command.
Vector store	Pinecone free tier	1 index, 100k vectors. More stable than self-hosted Chroma for a demo because it does not depend on the backend staying alive.

Component	Platform	Notes
Frontend	Netlify / GitHub Pages	Free static hosting for HTML/CSS/JS. Netlify preferred — auto-deploys from GitHub on push, supports environment variable injection for the API base URL.

Render free PostgreSQL instances expire after 90 days. This is acceptable for a portfolio demo. The seed script in the repo must cover full schema creation and sample data insertion so the database can be restored in one command: python seed.py